# AI LAB2

# Code:

```
% Main program with multiple runs
num_runs = 20; % Number of runs
num_generations = 2000; % Set number of generations
final_results = zeros(num_runs, 1); % Store the final best value for each run
all_best_values = zeros(num_generations, num_runs); % Store all best fitness values for each generation for all runs

for run = 1:num_runs
    % Main GA code begins
    % Initialize parameters
    pop_size = 300; % Increase population size to 300
    crossover_rate = 0.9; % Increase crossover probability
    mutation_rate = 0.3; % Increase mutation probability
    penalty_coefficient = 1000; % Constraint penalty coefficient
    shrink_value = 0.5; % Reduce mutation shrink value
    SUBPOP = 1; % Number of subpopulations, set to 1 indicating no subpopulation division

    % Define variable bounds FieldDR
    % The first row is the lower bound of each variable, and the second row is the upper bound
    FieldDR = repmat([-10; 10], 1, 7); % Bounds for 7 variables, range [-10, 10]

    % Define mutation options MutOpt, including mutation rate and shrink value
    MutOpt = [mutation_rate, shrink_value];

    % Step 1: Initialize population (create a real-valued population using crtrp with range [-10, 10])
    population = crtrp(pop_size, FieldDR);

    % Evolution main loop
    best_value = zeros(num_generations, 1); % To store the best fitness value of each generation
    for gen = 1:num_generations
        % Step 2: Fitness evaluation (call PrG9f function)
        fitness_values = PrG9f(population);

        % Step 3: Constraint evaluation and apply penalty (call PrG9c function)
```

```
        penalty_values = zeros(pop_size, 1);
        for i = 1:pop_size
            constraint_violations = PrG9c(population(i, :));
            % Calculate the total constraint violation
            total_violation = sum(max(0, constraint_violations));
            % Apply penalty
            penalty_values(i) = penalty_coefficient * total_violation;
        end

        % Final fitness values (with penalty)
        fitness_values = fitness_values + penalty_values;

        % Step 4: Fitness ranking
        ranked_fitness = ranking(fitness_values);

        % Step 5: Selection
        num_parents = pop_size; % Keep the number of parents equal to the population size
        selected_indices = rws(ranked_fitness, num_parents); % Use roulette wheel selection

        % Use the selected indices to get the complete individuals
        selected_parents = population(selected_indices, :);

        % Step 6: Crossover
        offspring = xovmp(selected_parents, crossover_rate); % Use multi-point crossover

        % Ensure that the number of columns of offspring matches the original population
        [num_offspring, num_variables] = size(offspring);
        if num_variables ~= size(FieldDR, 2)
            error('Error: The number of variables in offspring does not match FieldDR.');
        end

        % If the number of offspring is less than the original population, add more individuals
        if num_offspring < pop_size
            % Randomly select some individuals from the original population until reaching
the population size
            num_additional = pop_size - num_offspring;
            additional_indices = randi([1, pop_size], num_additional, 1);
            additional_individuals = population(additional_indices, :);

            % Ensure the dimensions match when concatenating
            offspring = [offspring; additional_individuals];
        elseif num_offspring > pop_size
            % If the number of offspring exceeds pop_size, randomly remove some
individuals
```

```
            offspring = offspring(1:pop_size, :);
        end

        % Step 7: Mutation (using mutbga, providing FieldDR and MutOpt parameters)
        mutated_offspring = mutbga(offspring, FieldDR, MutOpt);

        % Step 8: Reinsertion
        InsOpt = [1, 1.0]; % Use fitness-based insertion, insert all offspring
        population = reins(population, mutated_offspring, SUBPOP, InsOpt, fitness_values);

        % Record the best fitness value for each generation
        best_value(gen) = min(fitness_values);
    end

    % Record the final best value of each run
    final_results(run) = min(fitness_values);

    % Store the best value for each generation for each run
    all_best_values(:, run) = best_value;

    % Output the result of the current run
    fprintf('Run %d: Best Fitness Value = %f\n', run, final_results(run));
end

% Summarize the results of 20 runs
best_result = min(final_results);
worst_result = max(final_results);
mean_result = mean(final_results);
median_result = median(final_results);

% Print summary results
fprintf('\nSummary of 20 Runs:\n');
fprintf('Best Value: %f\n', best_result);
fprintf('Worst Value: %f\n', worst_result);
fprintf('Mean Value: %f\n', mean_result);
fprintf('Median Value: %f\n', median_result);

% Plot convergence trend using the median value of the 20 runs
median_convergence = median(all_best_values, 2);
figure;
plot(1:num_generations, median_convergence);
xlabel('Generation');
ylabel('Median Best Fitness Value');
title('Convergence Trend of GA Using Median Value of 20 Runs');
```

```
% Objective function: PrG9f
function y = PrG9f(x)
    % x: Population matrix, each row is an individual
    [N, ~] = size(x);
    y = zeros(N, 1);
    for i = 1:N
        xi = x(i, :);
        y(i) = (xi(1) - 10)^2 + 5 * (xi(2) - 12)^2 + xi(3)^4 + 3 * (xi(4) - 11)^2 + ...
                10 * xi(5)^6 + 7 * xi(6)^2 + xi(7)^4 - 4 * xi(6) * xi(7) - 10 * xi(6) - 8 * xi(7);
    end
end

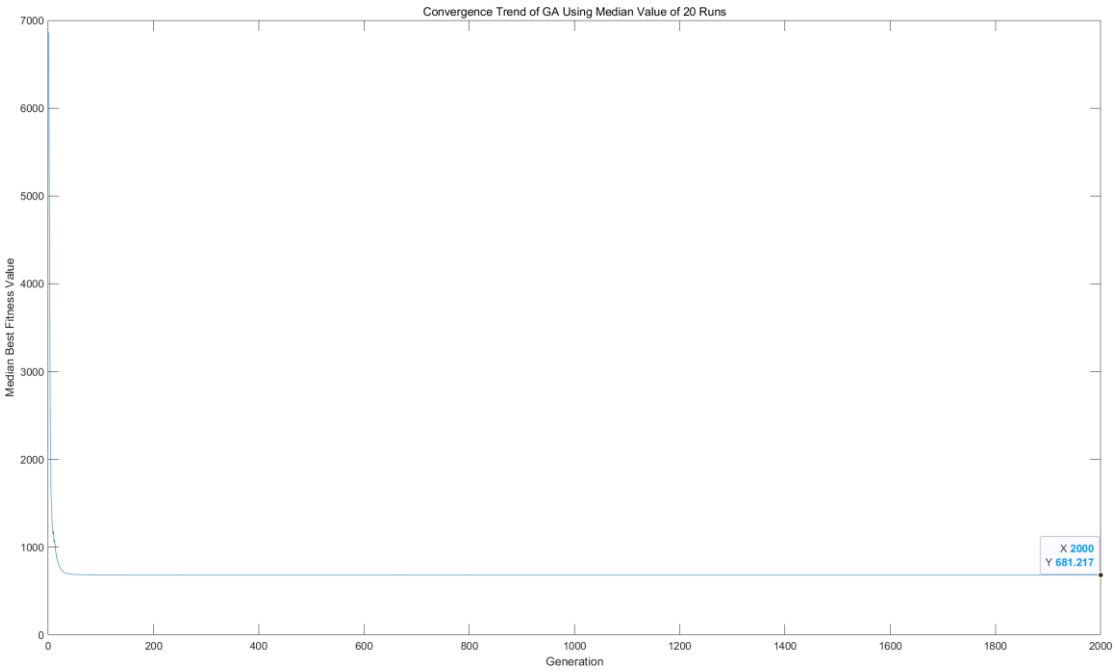
% Constraint function: PrG9c
function y = PrG9c(x)
    % x: A single individual (1x7 vector)
    % Constraints
    v1 = 2 * x(1)^2;
    v2 = x(2)^2;
    y(1) = v1 + 3 * v2^2 + x(3) + 4 * x(4)^2 + 5 * x(5) - 127;
    y(2) = 7 * x(1) + 3 * x(2) + 10 * x(3)^2 + x(4) - x(5) - 282;
    y(3) = 23 * x(1) + v2 + 6 * x(6)^2 - 8 * x(7) - 196;
    y(4) = 2 * v1 + v2 - 3 * x(1) * x(2) + 2 * x(3)^2 + 5 * x(6) - 11 * x(7);

    % Variable lower bounds
    for j = 1:7
        y(j + 4) = -x(j) - 10;
    end

    % Variable upper bounds
    for j = 1:7
        y(j + 11) = x(j) - 10;
    end

    % Transpose the constraint vector to a column vector
    y = y';
end
```

# Plot




# Table

# >> lab22

Run 1: Best Fitness Value = 681.542996

Run 2: Best Fitness Value = 682.667244

Run 3: Best Fitness Value = 681.842389

Run 4: Best Fitness Value = 680.994322

Run 5: Best Fitness Value = 680.877781

Run 6: Best Fitness Value = 681.030218

Run 7: Best Fitness Value = 682.483543

Run 8: Best Fitness Value = 681.924588

Run 9: Best Fitness Value = 680.922826

Run 10: Best Fitness Value = 680.777811

Run 11: Best Fitness Value = 681.007045

Run 12: Best Fitness Value = 681.266910

Run 13: Best Fitness Value = 681.055652

Run 14: Best Fitness Value = 680.921893

Run 15: Best Fitness Value = 683.300193

Run 16: Best Fitness Value = 681.376075

Run 17: Best Fitness Value = 681.262082

Run 18: Best Fitness Value = 681.436855

Run 19: Best Fitness Value = 681.171663

Run 20: Best Fitness Value = 681.074798

Summary of 20 Runs:

Best Value: 680.777811

Worst Value: 683.300193

Mean Value: 681.446844

Median Value: 681.216872

# explanation

1. Population Size (pop_size = 300)
    - A population size of 300 provides enough diversity while keeping computation reasonable. Increasing population size helps but only marginally improves the solution.
2. Number of Generations (num_generations = 2000)
    - 2000 generations allow enough iterations for convergence. Increasing generations further only yields diminishing improvements.
3. Crossover Rate (crossover_rate = 0.9)
    - A high crossover rate encourages effective recombination. This helps the algorithm explore the solution space and combine promising traits.
4. Mutation Rate (mutation_rate = 0.3)
    - Increasing the mutation rate from 0.1 to 0.3 greatly improved the ability to approach the ideal value. This higher rate helps introduce new variations and avoids getting stuck in local optima.
5. Penalty Coefficient (penalty_coefficient = 1000)
    - The penalty coefficient discourages constraint violations. A value of 1000 effectively penalizes infeasible solutions while allowing exploration near the boundaries.
6. Mutation Shrink Value (shrink_value = 0.5)
    - The shrink value reduces the mutation impact over time. It helps focus on exploration in early stages and fine-tuning later.
7. Number of Runs (num_runs = 20)
    - Running the GA 20 times provides consistency and robustness. This helps evaluate how well the GA performs on average.

Summary

- Increasing population size and number of generations improved the solution only slightly.
- Increasing mutation rate to 0.3 had the most significant impact on approaching the ideal value.
- The combination of high crossover rate, moderate mutation rate, and constraint handling via penalties provided a good balance between exploration, exploitation, and constraint satisfaction.